

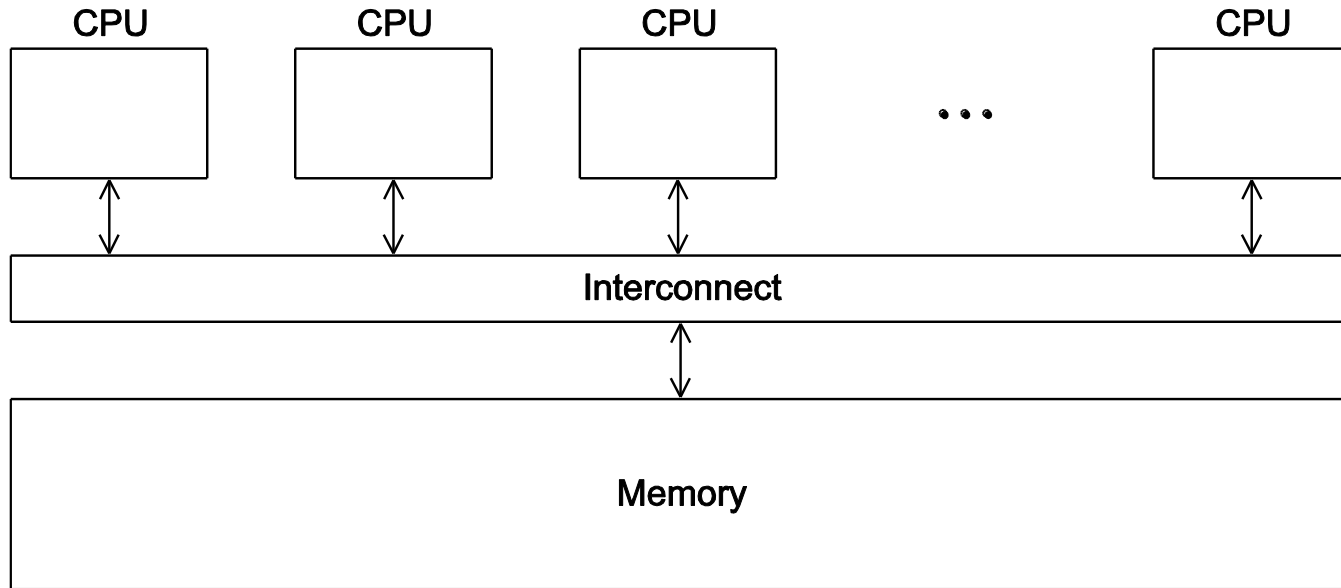
CPSC 375 High-Performance Computing

Lecture 20

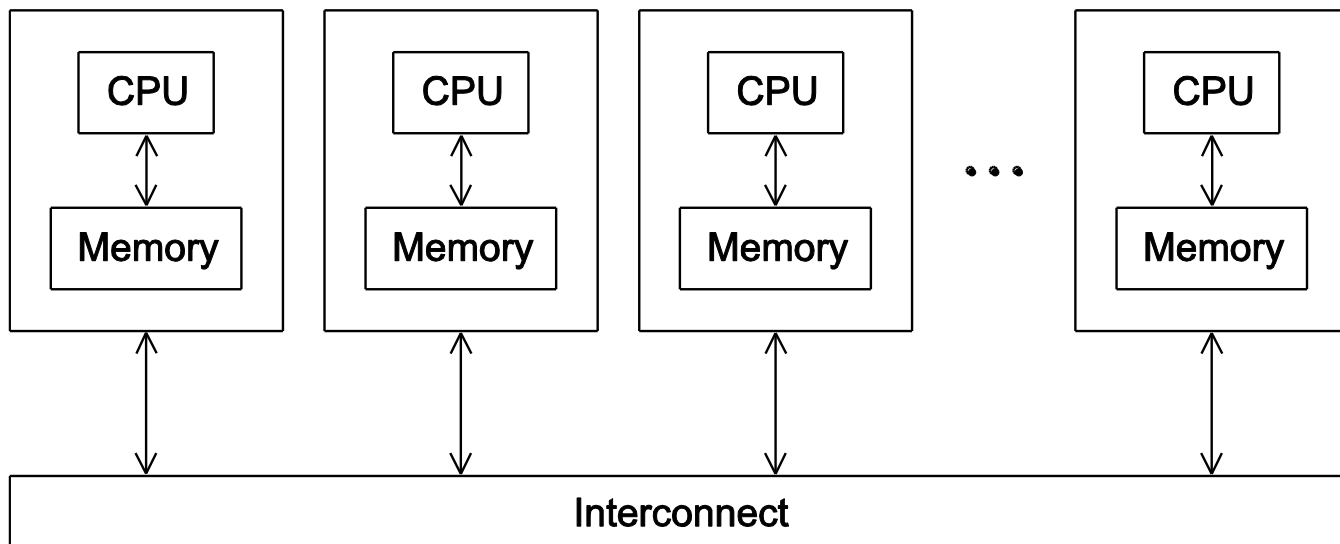
Distributed-Memory Programming

Introduction to MPI

A shared memory system



A distributed memory system



Processes

- Number is specified at start-up time.
- Remains constant throughout execution of program.
- All execute same program.
- Each has unique ID number.
- Alternately performs computations and communicates.

Advantages of message-passing model

- Gives the programmer the ability to manage the memory hierarchy.
- Portability to many architectures.
- Easier to create a deterministic program.
- Simplifies debugging.

The Message Passing Interface (MPI)

- Late 1980s: vendors had unique libraries
- 1989: Parallel Virtual Machine (PVM) developed at Oak Ridge National Lab
- 1992: Work on MPI standard begun
- 1994: Version 1.0 of MPI standard
- 1997: Version 2.0 of MPI standard
- 2012: Version 3.0 of MPI standard
- 2021: Version 4.0 of MPI standard
- Today: MPI is the dominant message passing library standard

Our first MPI program

```
#include <mpi.h>

MPI_Init(NULL, NULL);
MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

if (my_rank != 0) {
    sprintf(greeting, "Greetings from process %d of %d!", my_rank, comm_sz);
    MPI_Send(greeting, strlen(greeting)+1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
} else {
    printf("Greetings from process %d of %d!\n", my_rank, comm_sz);
    for (int q = 1; q < comm_sz; q++) {
        MPI_Recv(greeting, MAX_STRING, MPI_CHAR, q, 0, MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
        printf("%s\n", greeting);
    }
}

MPI_Finalize();
```

Compilation

wrapper script to compile *source file*

`mpicc -g -Wall -o mpi_hello mpi_hello.c`

produce debugging information *turns on all warnings* *create this executable file name (as opposed to default a.out)*

Execution

`mpiexec -n <number of processes> <executable>`

`mpiexec -n 1 ./mpi_hello`

 *run with 1 process*

`mpiexec -n 4 ./mpi_hello`

 *run with 4 processes*

Execution

```
mpiexec -n 1 ./mpi_hello
```

Greetings from process 0 of 1 !

```
mpiexec -n 4 ./mpi_hello
```

Greetings from process 0 of 4 !

Greetings from process 1 of 4 !

Greetings from process 2 of 4 !

Greetings from process 3 of 4 !

MPI Programs

- Written in C.
 - Has `main`.
 - Uses `stdio.h`, `string.h`, etc.
- Need to add `mpi.h` header file.
- Identifiers defined by MPI start with “MPI_”.
- First letter following underscore is uppercase.
 - For function names and MPI-defined types.
 - Helps to avoid confusion.

MPI Components

- **MPI_Init**

- Tells MPI to do all the necessary setup.

```
int MPI_Init(  
    int* argc_p /* in/out */,  
    char*** argv_p /* in/out */);
```

- **MPI_Finalize**

- Tells MPI to clean up anything allocated for this program.

```
int MPI_Finalize(void);
```

Basic Outline

```
#include <mpi.h>

int main(int argc, char* argv[]) {

    /* No MPI calls before this */
    MPI_Init(&argc, &argv);

    // ... Parallel code goes here ...

    MPI_Finalize();
    /* No MPI calls after this */

    return 0;
}
```

Communicators

- A collection of processes that can send messages to each other.
- `MPI_Init` defines a communicator that consists of all the processes created when the program is started.
- Called `MPI_COMM_WORLD`.

Communicators

```
int MPI_Comm_size(  
    MPI_Comm comm      /* in      */,  
    int* comm_sz_p /* out      */);
```



number of processes in the communicator

```
int MPI_Comm_rank(  
    MPI_Comm comm      /* in      */,  
    int* my_rank_p /* out      */);
```



my rank (the process making this call)

SPMD

- Single-Program Multiple-Data
- We compile one program.
- Process 0 does something different.
 - Receives messages and prints them while the other processes do the work.
- The **if-else** construct makes our program SPMD.

Communication

```
int MPI_Send(  
    void*          msg_buf_p    /* in */,  
    int            msg_size     /* in */,  
    MPI_Datatype   msg_type     /* in */,  
    int            dest         /* in */,  
    int            tag          /* in */,  
    MPI_Comm       communicator /* in */);
```

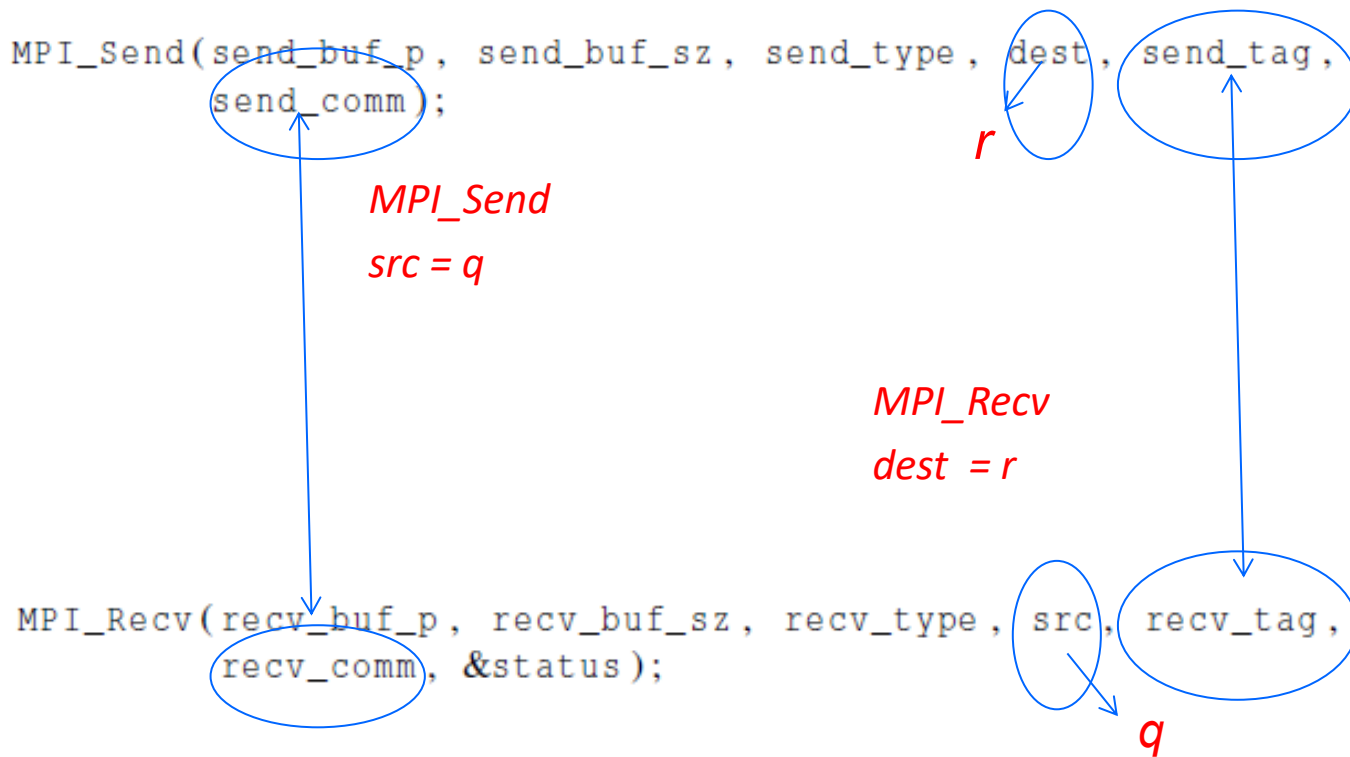
Data types

MPI Datatype	C Datatype	Description
MPI_CHAR	signed char	8-bit character
MPI_SHORT	signed short int	16-bit signed int
MPI_INT	signed int	Standard signed int
MPI_LONG	signed long int	32/64-bit signed int
MPI_LONG_LONG	signed long long int	64-bit signed int
MPI_UNSIGNED_CHAR	unsigned char	8-bit unsigned int
MPI_UNSIGNED	unsigned int	Standard unsigned int
MPI_FLOAT	float	Single-precision float
MPI_DOUBLE	double	Double-precision float
MPI_LONG_DOUBLE	long double	Extended-precision float
MPI_BYTE	—	8 bits (uninterpreted)
MPI_PACKED	—	Heterogeneous data

Communication

```
int MPI_Recv(  
    void*          msg_buf_p    /* out */,  
    int           buf_size      /* in  */,  
    MPI_Datatype   buf_type     /* in  */,  
    int           source        /* in  */,  
    int           tag           /* in  */,  
    MPI_Comm       communicator /* in  */,  
    MPI_Status*    status_p     /* out */);
```

Message matching



Receiving messages

- A receiver can get a message without knowing:
 - the amount of data in the message,
 - the sender of the message,
 - or the tag of the message.

```
MPI_Recv(recv_buf_p, recv_buf_sz, recv_type, src, recv_tag,  
         recv_comm, &status);
```

MPI_Status*

```
MPI_Status* status;  
status.MPI_SOURCE  
status.MPI_TAG
```

```
MPI_SOURCE  
MPI_TAG  
MPI_ERROR
```

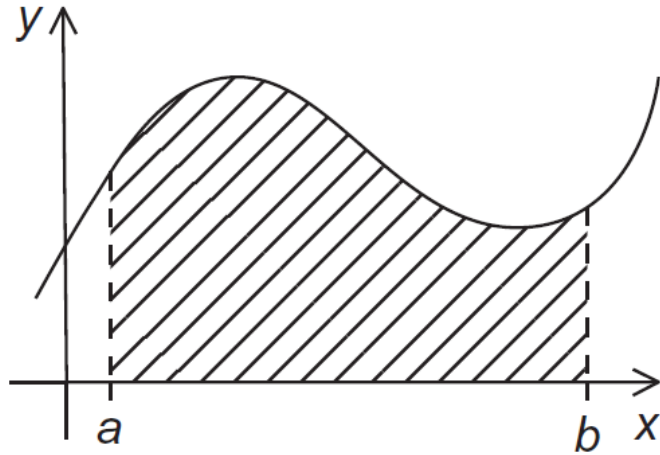
How much data am I receiving?

```
int MPI_Get_count(  
    MPI_Status*    status_p /* in */,  
    MPI_Datatype    type     /* in */,  
    int*           count_p   /* out */);
```

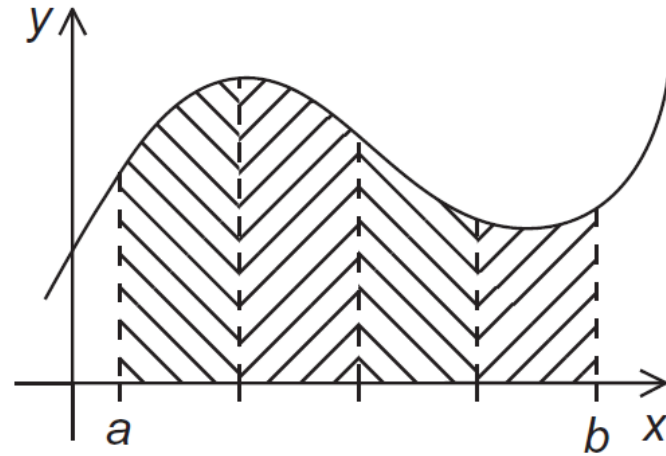
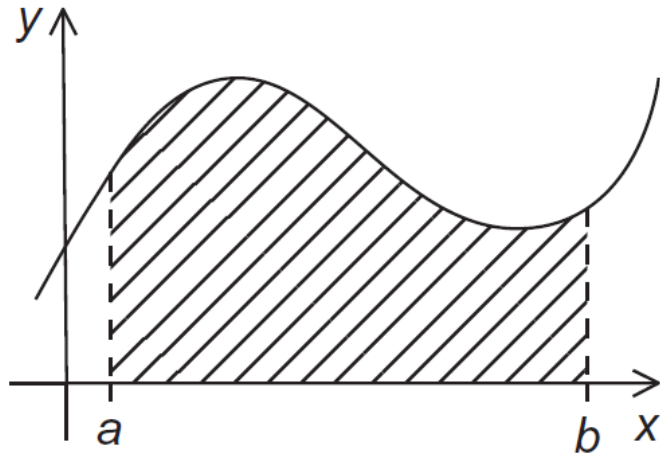
Issues with send and receive

- Exact behavior is determined by the MPI implementation.
- `MPI_Send` may behave differently with regard to buffer size, cutoffs and blocking.
- `MPI_Recv` always blocks until a matching message is received.

The Trapezoidal Rule Using MPI



The Trapezoidal Rule Using MPI



The Trapezoidal Rule

$$\text{Area of one trapezoid} = \frac{h}{2}[f(x_i) + f(x_{i+1})]$$

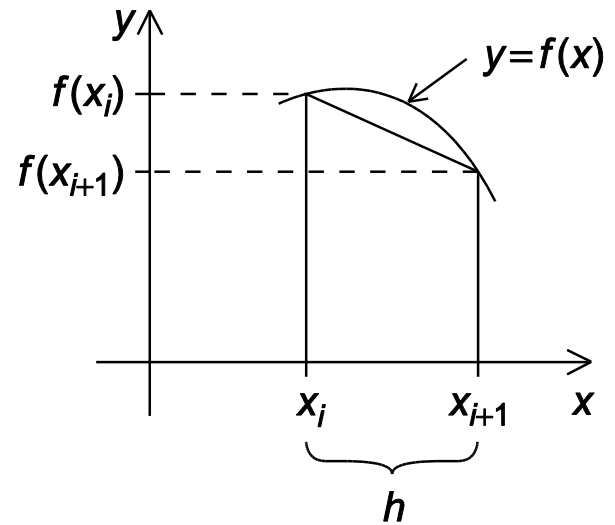
$$h = \frac{b - a}{n}$$

$$x_0 = a, x_1 = a + h, x_2 = a + 2h, \dots, x_{n-1} = a + (n - 1)h, x_n = b$$

$$\text{Sum of trapezoid areas} = h[f(x_0)/2 + f(x_1) + f(x_2) + \dots + f(x_{n-1}) + f(x_n)/2]$$

Pseudo-code for a serial program

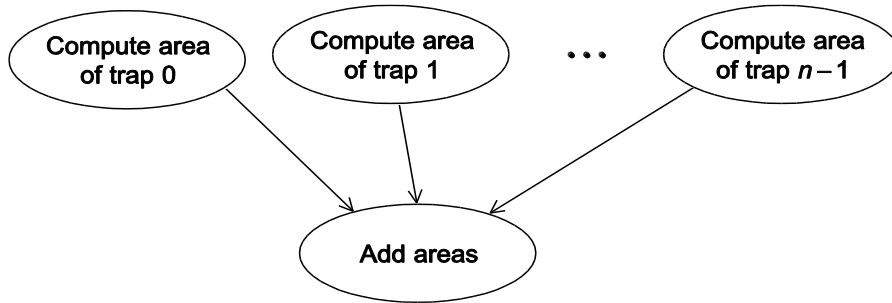
```
/* Input:  a, b, n */  
h = (b-a)/n;  
approx = (f(a) + f(b))/2.0;  
for (i = 1; i <= n-1; i++) {  
    x_i = a + i*h;  
    approx += f(x_i);  
}  
approx = h*approx;
```



Parallelizing the Trapezoidal Rule

1. Partition problem solution into tasks.
2. Identify communication channels between tasks.
3. Aggregate tasks into composite tasks.
4. Map composite tasks to cores.

Parallel pseudo-code



Get a, b, n ;

$h = (b-a)/n$;

$local_n = n/comm_sz$;

$local_a = a + my_rank * local_n * h$;

$local_b = local_a + local_n * h$;

$local_integral = Trap(local_a, local_b, local_n, h)$;

```
if (my_rank != 0)
```

```
    Send local_integral to process 0;
```

```
else /* my_rank == 0 */
```

```
    total_integral = local_integral;
```

```
    for (proc = 1; proc < comm_sz; proc++) {
```

```
        Receive local_integral from proc;
```

```
        total_integral += local_integral;
```

```
    }
```

```
if (my_rank == 0)
```

```
    print result;
```

Using MPI

```
int main(void) {  
    int my_rank, comm_sz, n = 1024, local_n;  
    double a = 0.0, b = 3.0, h, local_a, local_b;  
    double local_int, total_int;  
    int source;  
  
    MPI_Init(NULL, NULL);  
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);  
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);  
  
    h = (b-a)/n;      /* h is the same for all processes */  
    local_n = n/comm_sz; /* So is the number of trapezoids */  
  
    local_a = a + my_rank*local_n*h;  
    local_b = local_a + local_n*h;  
    local_int = Trap(local_a, local_b, local_n, h);  
  
    if (my_rank != 0) {  
        MPI_Send(&local_int, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);
```

```

} else {
    total_int = local_int;
    for (source = 1; source < comm_sz; source++) {
        MPI_Recv(&local_int, 1, MPI_DOUBLE, source, 0, MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
        total_int += local_int;
    }
}

if (my_rank == 0) {
    printf("With n = %d trapezoids, our estimate\n", n);
    printf("of the integral from %f to %f = %.15e\n", a, b, total_int);
}
MPI_Finalize();
return 0;
} /* main */

```

```

double Trap(
    double left_endpt /* in */,
    double right_endpt /* in */,
    int trap_count /* in */,
    double base_len /* in */) {
    double estimate, x;
    int i;

    estimate = (f(left_endpt) + f(right_endpt))/2.0;
    for (i = 1; i <= trap_count - 1; i++) {
        x = left_endpt + i*base_len;
        estimate += f(x);
    }
    estimate = estimate * base_len;

    return estimate;
} /* Trap */

```


Dealing with I/O

```
#include <stdio.h>
#include <mpi.h>

int main(void) {
    int my_rank, comm_sz;

    MPI_Init(NULL, NULL);
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);

    printf("Proc %d of %d > Does anyone have a toothpick?\n",
        my_rank, comm_sz);

    MPI_Finalize();
    return 0;
} /* main */
```

Running with 6 processes

Proc 0 of 6 > Does anyone have a toothpick?

Proc 1 of 6 > Does anyone have a toothpick?

Proc 2 of 6 > Does anyone have a toothpick?

Proc 4 of 6 > Does anyone have a toothpick?

Proc 3 of 6 > Does anyone have a toothpick?

Proc 5 of 6 > Does anyone have a toothpick?

Input

- Most MPI implementations only allow process 0 in `MPI_COMM_WORLD` access to `stdin`.
- Process 0 must read the data (`scanf`) and send to the other processes.

Function for reading user input

```
void Get_input(
    int  my_rank /* in */,
    int  comm_sz /* in */,
    double* a_p  /* out */,
    double* b_p  /* out */,
    int* n_p  /* out */) {
    int dest;

    if (my_rank == 0) {
        printf("Enter a, b, and n\n");
        scanf("%lf %lf %d", a_p, b_p, n_p);
        for (dest = 1; dest < comm_sz; dest++) {
            MPI_Send(a_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
            MPI_Send(b_p, 1, MPI_DOUBLE, dest, 0, MPI_COMM_WORLD);
            MPI_Send(n_p, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
        }
    } else { /* my_rank != 0 */
        MPI_Recv(a_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(b_p, 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Recv(n_p, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    }
} /* Get_input */
```

